



Esc

System Call Filtering

D3-SCF

D3-SCF (System Call Filtering)

Definition

Controlling access to local computer system resources with kernel-level capabilities.

Synonyms: *System Call Control*.

How it works

System call filtering uses a mandatory access control paradigm (that is, a non-discretionary access control) system because the rules and policies that determine access is determined by a security control authority and not distributed to local users. Access determinations are based on designed access control policies and are not based on local resource owner determinations.

Access is typically granted by defining sets of subjects and sets of objects. Subjects are the entities requesting access and objects are the resources that subjects are trying to access. Rules and policies are defined that associate subjects and object permissions and access controls.

Common implementations

Security label access control

A fine-grained form control is to apply security labels to individual resources, including processes, and the access control decisions are against a particular resource and a given user attempting to gain access. This type of control requires that the file system has built-in support for security labels.

Access controls are typically implemented through the use of label identifiers for every file system object. Identifier labels are applied to resources and users are assigned a similar access identifier. Users attempting to access a resource will result in the operating system performing an access control check. The access control check will compare the assigned user's credentials to that of the resource or object they are attempting to access.

A security context is associated with resources and is used to determine assess. Typical basic access control elements include users, roles and types and together they form a security context which is the basis for the security labels.

This type of access control is what is employed in SELinux [2]. This form of security kernel access control is considered the most flexible implementation, but it also is the most complex to deploy across the enterprise. Where multiple virtual machines (VM) are run together this type of access control is typically employed to ensure true isolation of processes and VMs.

File path level controls

A less fine-grained form of mandatory access control is to apply security labels that allow for access control at the file path level. Access control is filesystem agnostic and no relabeling of resources is required. Pathname access control usually seems more natural for implementation and corresponding access audits.

This type of system call filtering is what is employed in AppArmor [3]. AppArmor was developed to provide a simpler alternative method with much less management overhead. A simple access policy is maintained that defines path resource access rules. Access control attributes are typically associated with programs instead of users.

Considerations

Some implementations of security label-based control contain complex rules set that are hard to verify and complex to maintain over time.

Initial planning of access model and continuous monitoring of the available users, resources and object is necessary.

Implementations

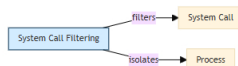
- Linux C-Groups, and policy engines like SELinux and AppArmor
- Windows Mandatory Integrity Control introduced in Windows Vista

Citations

- SELinux
- AppArmor

Artifact Relationships:

This defensive technique is related to specific artifacts. Click the artifact node for more information.



json

Technique Subclasses

There are 2 techniques in this category, **System Call Filtering**.

Name	ID	Definition	Synonyms
System Call Filtering	D3-SCF	Controlling access to local computer system resources with kernel-level capabilities.	System Call Control
- Local File Access Mediation	D3-LFAM	Local file access mediation is the process of an operating system granting or denying a specific access request to a local file.	Local File Access Control

Download ATT&CK Navigator Layer

Related ATT&CK Techniques:

These mappings are inferred, experimental, and will improve as the knowledge graph grows.

These offensive techniques are determined related because of the way this defensive technique, **d3f SystemCallFiltering**, **filters** **System Call**, and **isolates** **Process**.

Lateral Movement	Privilege Escalation	Collection	Discovery	Persistence	Execution	Credential Access	Defense Evasion
Use Alternate Authentication Material	Abuse Elevation Control Mechanism	Screen Capture	System Network Configuration Discovery	Event Triggered Execution	Windows Management Instrumentation	Exploitation for Credential Access	System Binary Proxy Execution
	Bypass User Account Control	Data Staged	Application Window Discovery	Netsh Helper DLL	Scheduled Task/Job	OS Credential Dumping	Mailinject
	Elevated Execution with Prompt	Local Data Staging	Software Discovery	AppInit DLLs	Scheduled Task	LSASS Memory	Control Panel
	Process Injection		Security Software Discovery	AppCert DLLs	Native API	LSA Secrets	Compiled HTML File
	YDSO		System Service Discovery	Server Software Component		Security Account Manager	Mahla
				Transport Agent			Rundl32

	ijacking		System Information Discovery			CMSTP
	Pirace System Calls			Web Shell		Abuse Elevation Control Mechanism
	Thread Execution Hijacking		Remote System Discovery	SQL Stored Procedures		
	Asynchronous Procedure Call		System Owner/User Discovery	Scheduled Task/Job	Credentials from Web Browsers	Bypass User Account Control
	Process Doppelg�nging		System Network Connections Discovery	Scheduled Task	Multi-Factor Authentication Request Generation	Elevated Execution with Prompt
	Dynamic-link Library Injection		System Time Discovery	Modify Authentication Process	Modify Authentication Process	Process Injection
	Thread Local Storage		Query Registry			VDSO Hijacking
	Access Token Manipulation		Process Discovery			Pirace System Calls
	Parent PID Spoofing		Virtualization/Sandbox Evasion			Thread Execution Hijacking
	Event Triggered Execution		Time Based Checks			Asynchronous Procedure Call
	Netsh Helper DLL					Process Doppelg�nging
	AppInit DLLs					Dynamic-link Library Injection
	AppCert DLLs					Thread Local Storage
	Scheduled Task/Job					Access Token Manipulation
	Scheduled Task					Parent PID Spoofing
						Use Alternate Authentication Material
						Decrypt/decode/Decode Files or Information
						Impair Defenses
						Disable or Modify Tools
						Modify Authentication Process
						XSL Script Processing
						Masquerading
						Match Legitimate Resource Name or Location
						Virtualization/Sandbox Evasion
						Time Based Checks

References

All Academic Paper Patent Internet Article

The following references were used to develop the System Call Filtering knowledge-base article.
(Note: the consideration of references does not imply specific functionality exists in an offering.)

Analysis of the Windows Vista Security Model

Reference Type: Academic Paper Organization: Symantec Corporation Author: Matthew Conover
Source:
https://web.archive.org/web/20140407025337/http://www.symantec.com/avcenter/reference/Windows_Vista_Security_Model_Analysis.pdf

Architecture of transparent network security for application containers

Reference Type: Patent Organization: Neuvector Inc Author: Gang Duan
Source:
<https://patents.google.com/patent/US20170093922A1>

Overview of the seccomp sandbox

Reference Type: Internet Article
Source:
<https://code.google.com/archive/p/seccompsandbox/wikis/overview.wiki>

close